
ENGINEERING REFERENCE

AI Safety Council

Technical Build Specification

Developer Handbook — Schema, APIs & Data Flows

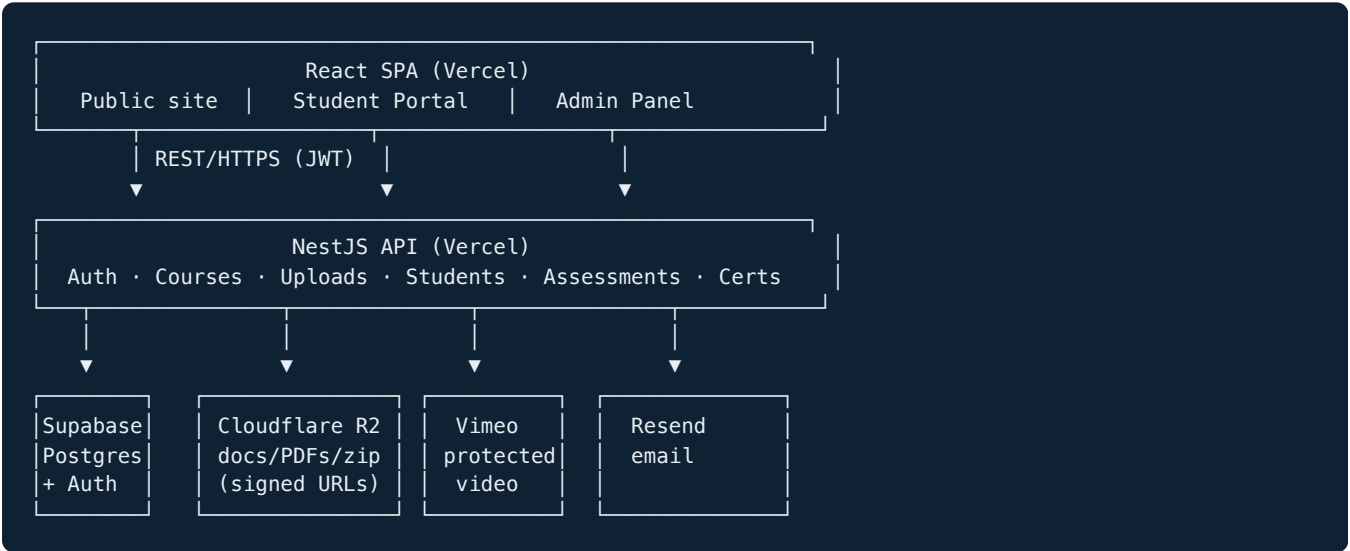
Certified AI Practitioner for OSH (AI-OSH)

AI Safety Council — Technical Build Specification (Developer Handbook)

Platform: AI Safety Council — AI-OSH LMS (MVP) **Audience:** Development team (build reference) **Stack:** React · NestJS · Supabase (Postgres + Auth + RLS) · Cloudflare R2 · Vimeo (protected video) · Vercel · Resend (email) **Frontend design:** built on a purchased/ready-made HTML template, customized to branding — not designed from scratch. **Team:** 2 developers **Date:** 01 July 2026

Single source of truth for **what to build and how the parts connect**: schema, API contracts, feature specs, the bulk-upload engine (§8), content protection (§9), and data flows. Pair with the *Project Overview & Costing* and *Build Plan* documents.

1. System Overview



Golden rules

- The **API is the only writer** to the database and R2. The SPA never holds privileged keys.
- **Supabase Auth** issues JWTs; **NestJS guards** enforce roles on every protected route; **RLS** in Postgres is the backstop.
- **Video lives on Vimeo** as a domain-restricted, download-disabled protected embed. **R2 stores everything else** (ZIP, notes, worksheets/PDFs, submissions, certificates) served via **short-lived signed URLs**.
- **Content protection** (watermarking, expiring links, disabled download) is a first-class feature — see §9.

2. Repository Structure (monorepo)

```
ai-safety-council/
├── apps/
│   └── web/                # React (Vite) — public + student + admin
```

```

├── src/{pages,features,components,lib,routes.tsx}
├── api/ # NestJS
│   ├── src/{modules,common,db,main.ts}
│   └── # modules: auth, courses, uploads, students, assessments,
│       # certificates, protection, public
├── packages/
│   └── shared/ # shared TS types + zod DTOs (imported by web & api)

```

3. Environments & Configuration

Env var	Used by	Purpose
<code>SUPABASE_URL</code>	api, web	Project URL
<code>SUPABASE_ANON_KEY</code>	web	Public client (auth only)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	api	Privileged DB access (server only)
<code>SUPABASE_JWT_SECRET</code>	api	Verify JWTs in guards
<code>R2_ACCOUNT_ID</code> / <code>R2_ACCESS_KEY_ID</code> / <code>R2_SECRET_ACCESS_KEY</code> / <code>R2_BUCKET</code>	api	R2 (S3-compatible)
<code>VIMEO_ACCESS_TOKEN</code>	api	Vimeo API (set privacy/domain lock, fetch embed)
<code>VIMEO_ALLOWED_DOMAIN</code>	api	Domain the videos are locked to
<code>RESEND_API_KEY</code> / <code>EMAIL_FROM</code>	api	Transactional email (SPF/DKIM verified sender)
<code>APP_BASE_URL</code>	api, web	Links in emails, LinkedIn share, watermark context
<code>WATERMARK_SECRET</code>	api	Sign/verify watermark tokens

Environments: **local** → **staging** → **production** (separate Supabase projects + R2 prefixes + Vimeo folders).

4. Database Schema (Postgres / Supabase)

4.1 Enums

```

create type user_role as enum ('admin', 'student');
create type course_status as enum ('draft', 'published', 'archived');
create type video_provider as enum ('vimeo', 'youtube', 'r2');
create type assessment_type as enum ('mcq', 'pdf');
create type submission_status as enum ('pending', 'under_review', 'graded', 'approved', 'rejected');
create type enrollment_status as enum ('active', 'completed');
create type certificate_status as enum ('pending', 'issued');
create type upload_status as enum ('uploaded', 'processing', 'completed', 'completed_with_errors', 'failed');

```

4.2 Tables

```
create table profiles (
  id                uuid primary key default gen_random_uuid(),
  auth_user_id      uuid unique references auth.users(id) on delete cascade,
  email             text not null unique,
  full_name         text not null,
  role              user_role not null default 'student',
  is_active         boolean not null default true,
  must_change_password boolean not null default true,
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now()
);

create table courses (
  id                uuid primary key default gen_random_uuid(),
  title             text not null,
  slug              text not null unique,
  subtitle          text,
  description       text,
  thumbnail_url     text,
  status            course_status not null default 'draft',
  created_by        uuid references profiles(id),
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now()
);

create table modules (
  id                uuid primary key default gen_random_uuid(),
  course_id         uuid not null references courses(id) on delete cascade,
  title             text not null,
  position          int not null,
  created_at        timestampz not null default now(),
  unique (course_id, position)
);

create table lessons (
  id                uuid primary key default gen_random_uuid(),
  module_id         uuid not null references modules(id) on delete cascade,
  title             text not null,
  position          int not null,
  video_provider    video_provider not null default 'vimeo',
  video_url         text,                -- Vimeo URL/ID (protected embed). null = attach later
  video_duration_s  int default 0,
  reading_notes     text,                -- markdown
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now(),
  unique (module_id, position)
);

create table resources (
  -- downloadable worksheets/PDFs per lesson
  id                uuid primary key default gen_random_uuid(),
  lesson_id         uuid not null references lessons(id) on delete cascade,
  title             text not null,
  file_url          text not null,      -- R2 key
  file_type         text,
  file_size_bytes   bigint,
  position          int not null default 0,
  created_at        timestampz not null default now()
);

create table enrollments (
  id                uuid primary key default gen_random_uuid(),
  student_id        uuid not null references profiles(id) on delete cascade,
  course_id         uuid not null references courses(id) on delete cascade,
```

```

status          enrollment_status not null default 'active',
assigned_by     uuid references profiles(id),
assigned_at     timestampz not null default now(),
completed_at    timestampz,
unique (student_id, course_id)
);

create table lesson_progress (
  id             uuid primary key default gen_random_uuid(),
  enrollment_id  uuid not null references enrollments(id) on delete cascade,
  lesson_id      uuid not null references lessons(id) on delete cascade,
  is_completed   boolean not null default false,
  last_position_s int not null default 0,
  completed_at   timestampz,
  updated_at     timestampz not null default now(),
  unique (enrollment_id, lesson_id)
);

create table assessments (
  id             uuid primary key default gen_random_uuid(),
  course_id      uuid not null references courses(id) on delete cascade,
  title          text not null,
  type           assessment_type not null,
  position       int not null default 0,
  pass_score     int,                -- % for mcq; null for pdf
  created_at     timestampz not null default now()
);

create table mcq_questions (
  id             uuid primary key default gen_random_uuid(),
  assessment_id  uuid not null references assessments(id) on delete cascade,
  question_text  text not null,
  options        jsonb not null,    -- [{ "id":"a","text":"..." }, ...]
  correct_option_id text not null,
  marks          int not null default 1,
  position       int not null default 0
);

create table assessment_submissions (
  id             uuid primary key default gen_random_uuid(),
  assessment_id  uuid not null references assessments(id) on delete cascade,
  student_id     uuid not null references profiles(id) on delete cascade,
  enrollment_id  uuid not null references enrollments(id) on delete cascade,
  type           assessment_type not null,
  status         submission_status not null default 'pending',
  score          numeric,
  max_score      numeric,
  answers        jsonb,             -- mcq: { "question_id":"option_id" }
  pdf_url        text,             -- R2 key for pdf submissions
  feedback       text,
  graded_by      uuid references profiles(id),
  submitted_at   timestampz not null default now(),
  graded_at      timestampz
);

create table certificates (
  id             uuid primary key default gen_random_uuid(),
  student_id     uuid not null references profiles(id) on delete cascade,
  course_id      uuid not null references courses(id) on delete cascade,
  enrollment_id  uuid references enrollments(id) on delete set null,
  certificate_url text,            -- R2 key of uploaded PDF
  status         certificate_status not null default 'pending',
  linkedin_share_url text,
  issued_by      uuid references profiles(id),
  issued_at      timestampz,
  created_at     timestampz not null default now(),

```

```

unique (student_id, course_id)
);

create table upload_jobs (
  id          uuid primary key default gen_random_uuid(),
  target_course_id uuid references courses(id) on delete set null, -- created draft course
  uploaded_by  uuid references profiles(id),
  zip_file_url text not null,      -- R2 key
  status       upload_status not null default 'uploaded',
  report       jsonb,              -- see §8.5
  created_at   timestampz not null default now(),
  completed_at timestampz
);

create table contact_messages (
  id uuid primary key default gen_random_uuid(),
  name text not null, email text not null, phone text, message text not null,
  created_at timestampz not null default now()
);

create table email_log (
  id uuid primary key default gen_random_uuid(),
  recipient text not null, type text not null, status text not null,
  provider_id text, created_at timestampz not null default now()
);

```

4.3 Indexes

`enrollments(student_id)` , `enrollments(course_id)` , `lesson_progress(enrollment_id)` ,
`assessment_submissions(status)` , `assessment_submissions(student_id)` ,
`modules(course_id,position)` , `lessons(module_id,position)` , `courses(status)` .

4.4 Row-Level Security (intent)

Table	Student	Admin
profiles	read own	all
courses/modules/lessons/resources	read only enrolled + published	all
enrollments / lesson_progress	read/write own	all
assessment_submissions	read/insert own; cannot set score/status	all
certificates	read own	all
contact_messages	insert (public)	read

Enforce in **both** Nest guards and RLS.

5. Authentication & Authorization

- Supabase Auth (email + password), **no public sign-up**.
- Admin creates students via Supabase Admin API → inserts `profiles` (`must_change_password=true`)
→ emails credentials.

- Login → JWT → `Authorization: Bearer` to API. Guards: `JwtAuthGuard` + `RolesGuard` (`@Roles('admin'|'student')`).
 - First login forces password change.
-

6. API Contract (REST) — base `/api/v1`

Auth: `GET /auth/me` · `POST /auth/change-password` · `POST /auth/logout` **Public:** `GET /public/courses` · `POST /public/contact` **Admin — content:** `POST/GET /admin/courses` · `GET/PATCH/DELETE /admin/courses/:id` · `POST /admin/courses/:id/publish` · `module/lesson/resource` **CRUD** (`/admin/modules` , `/admin/lessons` , `/admin/resources`) **Admin — bulk upload:** `POST /admin/uploads/presign` · `POST /admin/uploads` · `POST /admin/uploads/:id/process` · `GET /admin/uploads/:id` (status + report) **Admin — students:** `POST/GET /admin/students` · `PATCH /admin/students/:id` · `POST /admin/students/:id/enrollments` · `DELETE /admin/enrollments/:id` **Admin — assessments:** assessment + MCQ **CRUD** · `GET /admin/submissions?status=under_review` (queue) · `PATCH /admin/submissions/:id/grade` **Admin — certificates:** `POST /admin/certificates` (upload + issue + email) · `GET /admin/certificates` **Student:** `GET /student/courses` · `GET /student/courses/:id` · `GET /student/lessons/:id` (Vimeo embed config + signed resource URLs + watermark token) · `POST /student/lessons/:id/progress` · `GET /student/assessments/:id` · `POST /student/assessments/:id/submit` · `GET /student/certificates` · `GET /student/certificates/:id/download`

7. Feature-by-Feature (purpose · data · endpoints · frontend · communicates with · edge cases)

- **F1 Public site** — reads published `courses` ; writes `contact_messages` ; emails admin. Edge: spam (rate-limit + honeypot).
 - **F2 Auth & roles** — `profiles` + `auth.users` ; login + forced password change; guards on all private routes. Edge: deactivated account, expired JWT.
 - **F3 Admin course/content CRUD** — `courses/modules/lessons/resources` ; nested editor + reorder; also the fix-up tool for bulk-upload output. Edge: position conflicts (transactional), can't publish empty course.
 - **F4 Bulk upload engine** — see the dedicated deep-dive in §8.
 - **F5 Student portal** — reads enrolments + course tree; writes `lesson_progress` ; renders **Vimeo protected player** (§9) + signed resource links. Edge: not enrolled → 403; resume position.
 - **F6 Assessments** — MCQ auto-grade (server-side) + PDF non-blocking review queue; PDFs to R2. Edge: resubmission policy, answer tampering (grade server-side only).
 - **F7 Certificates** — admin upload → issue → email + dashboard + LinkedIn. Edge: unique per student+course, re-issue/replace.
 - **F8 Email** — Resend; `email_log` . Edge: delivery failure (log/retry), unverified domain.
 - **F9 Content protection** — see §9: watermarking, expiring links, Vimeo domain-lock, disabled download.
-

8. Bulk Upload Engine — Detailed

8.1 Core principle: keep video OUT of the ZIP

A course is ~8h of video (~6–8 GB) — too large to push through a browser+server pipeline reliably.

Everything else in a course is small (< ~100 MB). So the ZIP carries **structure + notes + worksheets + video links only**. Videos are hosted on Vimeo and referenced by URL. This keeps the ZIP a few MB and makes the whole feature reliable.

8.2 The template (what the admin fills in)

A folder tree with tiny metadata files. We ship a pre-filled sample.

```
my-course/
├── course.json                # { "title": "...", "subtitle": "...", "description": "..." }
├── 01 Foundations/           # module (numeric prefix = order)
│   ├── module.json           # { "title": "AI Foundations" }
│   ├── 01 What is AI/        # lesson (numeric prefix = order)
│   │   └── lesson.json       # { "title": "What is AI?", "video": "https://vimeo.com/...",
│   │   └── "duration": 600 }
│   │       ├── notes.md      # reading notes (optional)
│   │       └── worksheets/   # optional PDFs
│   │           └── checklist.pdf
│   └── 02 Generative AI/
│       └── lesson.json
└── 02 Responsible AI/
    └── ...
```

Rules: ordering comes from the numeric prefixes; **video** may be a Vimeo URL **or** left empty (attach later in the editor); **notes.md** and **worksheets/** are optional.

8.3 Upload path (avoids server body limits)

1. Admin selects folder → browser zips it (or uploads a pre-zipped file).
2. POST /admin/uploads/presign → API returns a presigned R2 PUT URL.
3. Browser uploads the ZIP DIRECTLY to R2 (never through the API body).
4. POST /admin/uploads { zip_file_url } → creates upload_jobs(status=uploaded).
5. POST /admin/uploads/:id/process → runs the parser (§8.4) as a short background job.
6. Admin polls GET /admin/uploads/:id → status + report (§8.5).

Because the ZIP is small, processing fits comfortably in a single background run; no chunking needed.

Enforce a hard cap (e.g. **200 MB**) and reject with a clear error.

8.4 Parsing algorithm

```
processUpload(jobId):
  set status = processing
  download ZIP from R2 → extract to temp (use a streaming unzip lib: unzipper/yauzl)
  read & validate course.json → create courses(status='draft') [in a DB transaction]
  for each module folder (sorted by numeric prefix):
    validate module.json → insert modules(position = index)
    for each lesson folder (sorted by numeric prefix):
      validate lesson.json → insert lessons(position = index,
        video_provider='vimeo', video_url=lesson.video || null,
        reading_notes = contents of notes.md if present)
    for each file in worksheets/:
      validate is PDF & size → upload to R2 → insert resources(...)
```



```
collect a per-item error instead of aborting on any single failure
set target_course_id, status = completed | completed_with_errors, report = {...}
(on fatal error: status = failed, report.error = message)
```

- **Draft-only:** a bulk upload always creates a **new draft course**. The admin reviews and publishes. Re-uploading creates another draft (no destructive upsert in MVP).
- **Transactional structure:** wrap course/module/lesson inserts so a fatal parse error leaves no half-course. File (R2) uploads happen after row creation; orphan-cleanup on failure.

8.5 Validation rules & report shape

Validation: `course.json` / `module.json` / `lesson.json` present & well-formed; titles non-empty; numeric prefixes unique per level; `video` is a valid Vimeo URL or empty; worksheets are PDFs under size cap.

```
{
  "created": { "modules": 8, "lessons": 40, "resources": 35 },
  "warnings": [
    { "path": "02 Responsible AI/03 Privacy", "issue": "no video link – attach later" }
  ],
  "errors": [
    { "path": "05 Leadership/01 Roadmap/worksheets/plan.docx", "issue": "not a PDF – skipped" }
  ]
}
```

The frontend renders this as a checklist with a deep link into F3 (course editor) to fix each item.

8.6 Two-step video attach

Videos can be added two ways, both supported:

- **In the template:** paste the Vimeo URL into `lesson.json` `video` .
- **After upload:** leave it empty; the lesson is created with `video_url=null` ; the admin pastes/attaches the Vimeo link per lesson in the editor. Lessons with no video show a "video pending" state (not shown to students until set).

9. Content Protection — Detailed

Tier: "Practical protection" (agreed). No DRM. Goal: block casual download/sharing and make any leak traceable. **Screenshots/screen-recording cannot be fully blocked on the web — do not claim otherwise.** Watermarking is the mitigation.

9.1 Video (Vimeo)

- Upload videos to Vimeo; set **privacy = "hide from Vimeo" + domain-level embed restriction** (`VIMEO_ALLOWED_DOMAIN`) so the embed only plays on our site.
- **Disable download** in Vimeo settings; hide unnecessary player controls.
- Serve the embed only to authenticated, enrolled students (checked in `GET /student/lessons/:id`).

9.2 Watermarking (the key deterrent)

- **On video:** overlay a semi-transparent HTML layer above the player showing the student's **name + email + ID**, repositioned periodically (e.g., every 20–30s move to a new corner). It appears in any

screenshot/screen-recording. Note: an overlay is a deterrent (removable via devtools by experts) — combine with §9.3.

- **On PDFs (stronger):** stamp the watermark **server-side into the PDF** before serving (e.g., pdf-lib) — student name/email tiled faintly across pages. This is baked into the file, not just displayed. Cache stamped copies per student in R2.

9.3 Links & documents (R2)

- All R2 objects private. Serve via **short-lived signed URLs** (e.g., 5–15 min), generated per authenticated request.
- Disable right-click / download attributes on the player and note viewers; disable text selection on notes (CSS `user-select:none` + no print styles).
- Never expose a permanent public file URL.

9.4 Access gating

- Every content endpoint verifies: valid JWT → correct role → **active enrolment** in the course that owns the lesson. No enrolment, no signed URL, no embed.

9.5 Watermark token

- `GET /student/lessons/:id` returns a signed **watermark token** (name/email/id, short TTL, signed with `WATERMARK_SECRET`) the frontend renders — so the identity in the overlay is server-vouched, not client-spoofable.

10. Critical Data Flows

MCQ (auto): submit → load `mcq_questions` (answers server-side) → compute score → insert `assessment_submissions(status=graded)` → return instantly.

PDF (non-blocking): upload → R2 → insert submission(`under_review`) → **student continues** → admin `PATCH /grade` → status graded/approved → email student.

Certificate: admin upload PDF → R2 → `certificates(status=issued, ...)` → email → student dashboard download (signed URL) / LinkedIn share.

Bulk upload: see §8.3–§8.5.

11. Storage Design (Cloudflare R2)

```
uploads/zips/{uploadJobId}.zip
courses/{courseId}/../resources/{resourceId}-{filename}.pdf
submissions/{studentId}/{submissionId}.pdf
certificates/{studentId}/{courseId}.pdf
watermarked/{studentId}/{resourceId}.pdf      # cached, stamped copies
```

All private; signed URLs only. **No video in R2** (Vimeo hosts video). Browser uploads (ZIP, worksheets, submission PDFs) use **presigned PUT** so large bodies never hit the API. R2 egress is free → cheap even at scale.

12. Course Size & Storage Sizing

Item	Where it lives	Typical size
Course video (~8h, ~40 lessons)	Vimeo (not R2)	~6–8 GB per course
Notes (markdown)	Postgres	KBs
Worksheets/PDFs	R2	< ~100 MB per course
Bulk-upload ZIP (no video)	R2 (transient)	a few MB – tens of MB
Student PDF submissions	R2	~1–5 MB each
Certificates	R2	~200 KB each

Guidance: keep per-video files reasonable (720p/1080p). Vimeo plan storage (Pro ≈ 1 TB) holds 100+ hours → many courses. Your R2 footprint stays small (docs only), which is why running cost is nearly flat with scale.

13. Non-Functional Requirements

Area	Requirement
Validation	zod/class-validator DTOs (shared package); reject unknown fields.
Authorization	Guard on every route + RLS; enrolment check before any content/signed URL.
Security	Signed URLs only; service-role key server-side; rate-limit auth/contact/upload; size + MIME checks on uploads.
Protection	Vimeo domain-lock + watermarking + expiring links (§9).
Error handling	Consistent <code>{ error: { code, message } }</code> ; bulk job never aborts whole batch on one item.
Idempotency	Progress + submissions safe to retry (unique constraints).
Observability	Request logs, <code>email_log</code> , upload job reports; add error monitoring (Sentry) from the Scaling stage.
Performance	Paginate lists; index FKs.
Responsive/A11y	Mobile-first; keyboard-navigable player and forms.

14. Build Order

Sequenced to avoid blockers; matches the phases in the *Build Plan*.

1. **Schema + Auth + RBAC** (foundation) → 2. **Admin course/content CRUD** → 3. **Bulk upload (§8) + student accounts** → 4. **Student portal + Vimeo protected video** → 5. **Assessments** → 6. **Certificates + email** → 7. **Content protection hardening (§9)** → 8. **QA, security, deploy, handover**.

Lock the **bulk-upload template (§8.2)** with the client before step 3 — highest-risk dependency. The frontend uses a customized ready-made HTML template as the design base (see header).

15. Definition of Done (per feature)

- DTO-validated, guarded, RLS-covered.
- Works for both roles with correct access control.
- Files land in the correct R2 path; no orphans; served via signed URLs.
- Content endpoints enforce enrolment + return watermark token where applicable.
- Handles the edge cases in §7/§8.
- Responsive + smoke-tested on staging before merge.